MMA3001 Project Report

Gap Reconstruction for Building Temperature Sensors: Interpolation vs Machine Learning

Author: Abhinav Jain
Student ID: 34321446
Unit: MMA3001 Numerical Methods and Machine Learning
Semester: 2026 S2
Repository: https://github.com/ajai0042-monash/MMA3001Repo

Table of Content

1. Engineering Problem....2
2. Inputs, Outputs and Data....2
  2.1 Input data....2
  2.2 Outputs....3
  2.3 Handling of invalid, missing, or unsupported inputs....3
  2.4 Data audit....3
3. Computational Approach....3
  3.1 Baseline....3
  3.2 Interpolation methods (Used From Week 3)....3
  3.3 Machine learning methods (Used From Week 5)....4
  3.4 Windowed ML wrapper (original contribution)....4
  3.5 Why these methods and known limitations....4
4. Validation Method and Results....4
  4.1 Validation design....4
  4.2 Metrics....5
  4.3 Physical plausibility check....5
  4.4 Engineering impact check (degree-hours)....5
  4.5 Transfer validation on pressure....6
  4.6 Results summary....6
  4.7 What the validation cannot establish....7
  4.8 Did validation influence model selection....7
5. Comparison and Optimisation Work....8
  5.1 Multi-metric comparison....8
  5.2 FLOPS estimates....8
  5.3 Optimisation: windowed ML....8
  5.4 Trade-offs and justification....9
6. Limitations and Lessons Learned....9
  Limitations....9
  Lessons learned....9
7. AI-Use Reflection....9
8. References....10

1. Engineering Problem

Modern buildings run on sensor data. HVAC systems decide when to cool or heat a room based on temperature readings every few minutes, comfort assessments are made from historical records, and energy audits add up thermal exposure over days or weeks. If the sensor drops out for an hour or a day, that gap has to be filled before the data can be used downstream. The usual approach is to draw a straight line between the last known reading and the next one, but this choice is rarely tested against anything, and I wanted to know how much it actually matters.

The specific problem I set out to answer is: for a building temperature sensor, how should missing data be filled, and does the choice of method change engineering decisions made from that data. The problem is relevant to mechanical engineering because comfort control, chiller sizing, energy budgeting, and compliance reporting all rely on cumulative temperature exposure, and these quantities are sensitive to how the gaps are filled. In this project, filling a single 24-hour gap with linear interpolation underestimated degree-hours of thermal exposure by 91%, which is enough to flip a comfort compliance decision in the wrong direction.

The project uses the Monash Smart Infrastructure Environmental Data, which contains readings from five sensors across the Clayton campus over roughly three years. The intended use of my solution is deciding, for a given gap length, which reconstruction method to apply before the data is used for comfort analysis. It is not meant to be a real-time filler during live operation, and it does not try to predict future values. The scope is also limited to one variable (temperature) on one sensor for the main study, with a short transfer check on pressure. A full multi-sensor spatial model is out of scope for a numerical methods unit.

2. Inputs, Outputs and Data

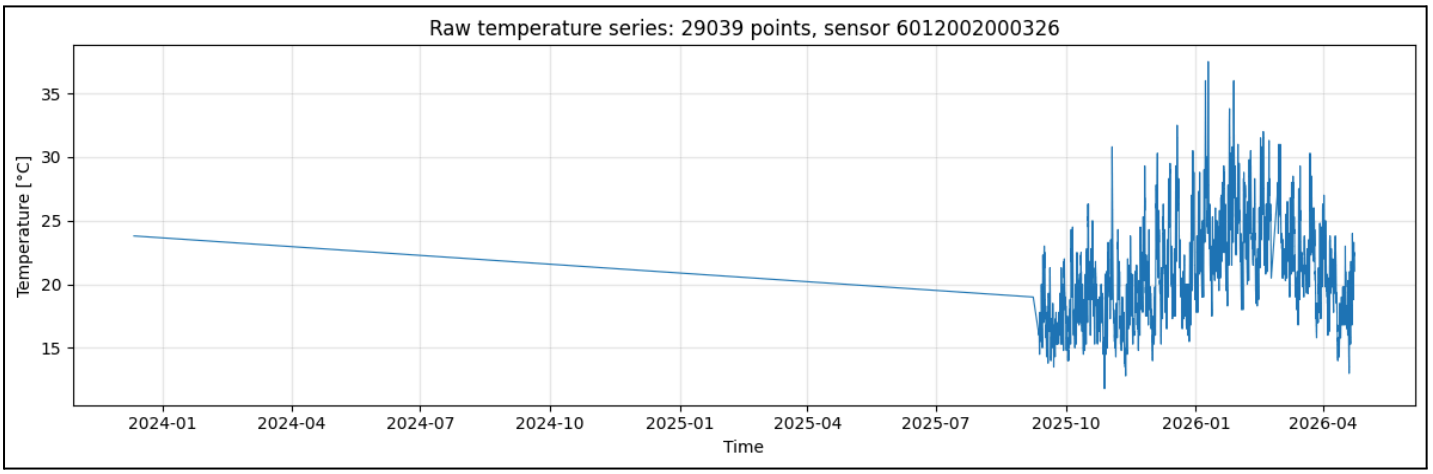


Fig 1: Raw Temperature Data from Sensor 6012002000326

2.1 Input data

The main input is a CSV file provided by the unit containing 179,447 rows of sensor data. Each row has a sensor ID, a nested JSON field with the actual readings, and metadata. The JSON unpacks into seven variables per reading: temperature in degrees Celsius, humidity in percent, CO_2 in ppm, pressure in millibars, battery voltage, light volatile organic compounds in ppb, and PM2.5 in micrograms per cubic metre. Timestamps are stored as Unix seconds and converted to Melbourne local time during loading. The range of valid values is bounded by physical plausibility: temperature 5–50 °C, CO2 350–5000 ppm, humidity 0–100%, pressure 950–1050 mb, LVOC 0–10 000 ppb, PM2.5 0–1000 µg/m³. Readings outside these bounds are removed during the audit.

2.2 Outputs

The pipeline produces a filled temperature series with no missing values, three error metrics per method per gap (MAE, RMSE, R2), residual and reconstruction overlay plots for visual inspection, degree-hours of thermal exposure above a comfort threshold, runtime per method, and FLOPS estimates for the interpolation methods. The error table has 54 rows total (6 methods x 3 gap lengths x 3 random placements) and feeds into the final method recommendation.

2.3 Handling of invalid, missing, or unsupported inputs

Several checks run during data loading. Rows with corrupted JSON return an empty dictionary and are skipped. Rows missing a timestamp or the target variable are dropped. Readings outside the physical bounds are filtered out and the count is printed. Duplicate timestamps are removed by keeping the first. All of this happens inside the **audit_data** and **get_sensor_series** functions in **data_loader.py**. Before I added these checks, three sensors appeared to have thousands of readings but the data was unusable.

2.4 Data audit

The audit on the raw CSV showed 179,447 rows across 5 unique sensor IDs, with the dataset covering May 2023 to April 2026. Missing values were present in LVOC (16.5%) and PM2.5 (1.0%) but temperature was complete. The real problem was timestamp integrity. Three of the five sensors (IDs ending 0241, 0227, 0777) had firmware timestamp corruption where every reading collapsed to a single datetime value. These sensors look complete from a row count perspective but are useless for time-series work. This left sensor **6012002000326**, which had 29,039 valid temperature readings. The valid data was concentrated after September 2025, so I filtered to that dense period (29,038 points at 10-minute median sampling) for all analysis. This filtering was a direct result of the audit.

3. Computational Approach

I implemented six reconstruction methods, grouped into three categories.

3.1 Baseline

For the baseline I used scikit-learn's DummyRegressor with the mean strategy. The brief asks for at least one baseline in the alternatives section, and this one just predicts the mean of all the known values at every gap position. It's basically useless in practice but it gives me a floor the other methods have to beat. If one of my methods can't beat the mean, that means something is wrong with my methodology.

3.2 Interpolation methods (Used From Week 3)

I used three interpolation methods from scipy. **Linear** (np.interp) just draws a straight line between the two known points on either side of a gap. **Cubic Spline** (scipy.interpolate.CubicSpline) fits a smooth piecewise cubic through all the known points using natural boundary conditions. **PCHIP** (scipy.interpolate.PchipInterpolator) is a shape-preserving version of cubic interpolation, meaning it won't overshoot between monotonic points like a regular cubic spline sometimes does.

All three need numeric x-values, so I convert the datetime index to seconds first. The assumption behind all three is that the signal between known points can be approximated by something simple locally. That works fine for short gaps but not so much for a whole day, because the temperature cycle moves too much.

3.3 Machine learning methods (Used From Week 5)

For the ML side I used **SVR** with an RBF kernel and a **small MLP neural network**. Both are from scikit-learn and I wrapped both in a Pipeline with StandardScaler as the first step so the features get normalised before fitting. For SVR I used C=10, epsilon=0.5, gamma="scale". For the MLP I used two hidden layers (64, 32), ReLU activation, Adam solver, max 500 iterations with early stopping on.

Both models take cyclic temporal features: $\sin(2\pi \cdot \text{hour}/24)$, $\cos(2\pi \cdot \text{hour}/24)$, and the same pair for weekday. I used sine/cosine encoding because hour 23 is actually close to hour 0, and a raw integer wouldn't capture that the model would think they were far apart.

The scaler goes first in the pipeline mainly because SVR's RBF kernel uses distances between points, and if the features are on different scales the big ones dominate the distance. Same reason helps the MLP as Adam makes assumptions about step sizes that work better on scaled inputs.

3.4 Windowed ML wrapper (original contribution)

This part is my own idea. When I first ran SVR with the full 29,000-point series it took about 50 seconds per gap, which was way too slow. I realised the model doesn't actually need to see every point going back months to learn the daily cycle around a gap. So I wrote a wrapper that restricts training to a 48-hour window around each gap.

With the wrapper, SVR drops to about 15 milliseconds per gap. That's roughly a 3000x speedup. And also, accuracy got better too, probably because the model wasn't being pulled by old seasonal variation that had nothing to do with the gap it was trying to fill.

3.5 Why these methods and known limitations

The interpolation methods came from Week 3 and they're the standard options for univariate time-series gap filling. SVR and MLP came from Week 5 and represent two different approaches to regression (kernel vs. neural). The windowing was my own idea once I saw how slow the global ML version was.

Things I didn't do that I probably should mention: cubic spline is known to overshoot on long gaps (which I confirmed in the results), I didn't run a grid search for SVR hyperparameters so my C and epsilon are just reasonable defaults from the weekly example, and my ML models only see time-of-day features. Adding readings from the other sensors or external weather data would be the obvious next step.

4. Validation Method and Results

4.1 Validation design

My approach was to use known ground truth. Since the data is complete across the dense period, I could just hide blocks of real values on purpose, then let each method try to reconstruct them and compare what it predicted against what I knew the real values were. This is basically the train/test split idea from week 5 but applied to time-series gaps instead of random points, which makes more sense here because random splits don't really work on time-series data (neighbouring points are too similar).

For the gap sizes I picked 1 hour, 6 hours, and 24 hours. These weren't arbitrary since they match realistic ways a sensor actually fails in a building: a brief network dropout, a half-day outage, or a full-day failure where someone has to fix it. I did 3 random placements per duration so I had 9 scenarios in total. That felt like enough that one

unlucky gap position wouldn't skew the overall ranking, but not so many that it took a long time to run. The random seed is fixed so if someone else runs the notebook they get the same gaps and same results.

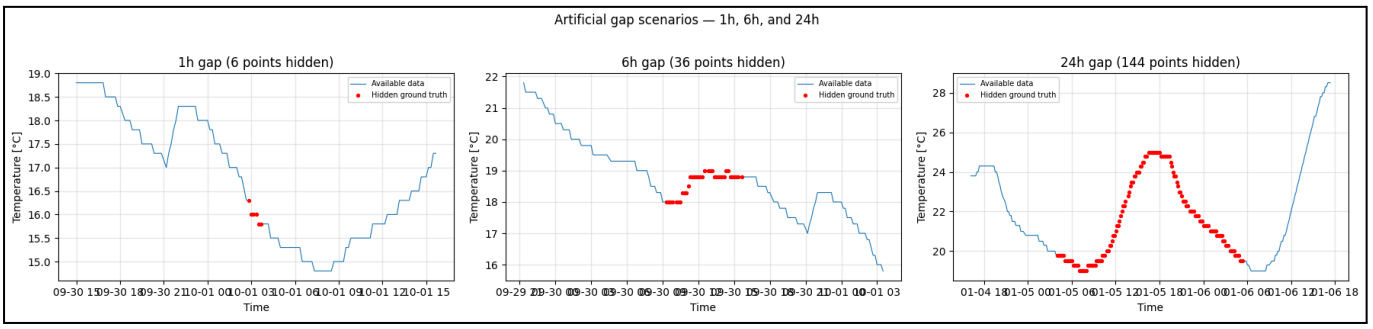


Fig 2: Gap Scenarios for 1hr, 6hr, 24hr

4.2 Metrics

I used the three metrics from Week 5. MAE (mean absolute error) is my primary ranking metric because it's directly readable in degrees Celsius, so "MAE 0.5" means the average prediction is off by half a degree. RMSE penalises big errors more than MAE does, so it matters when a method occasionally produces an illogical value with cubic spline overshoots are the clearest example of that.

R2 is reported for completeness but treated as a diagnostic rather than a ranking metric. The reason is that on short gaps the ground truth is nearly constant (temperature barely moves over an hour), and because R2 is defined relative to the variance of the target, any method with even a small absolute error can end up with a negative R2 while the "predict the mean" baseline looks better by default.

4.3 Physical plausibility check

Error metrics on their own can hide reconstructions that are physically absurd, so I included a dT/dt check as a sanity test. The idea is simple: indoor temperature doesn't change faster than about 2 °C per minute in real life, so if a reconstruction is swinging faster than that it has to be wrong even if its average error looks okay.

I used numpy.gradient on the filled series with real time coordinates to estimate the maximum rate of change for each method on each gap, then flagged anything above the 2 °C/min threshold. All five methods passed on both the 6h and 24h gaps — the biggest dT/dt I saw was 0.175 °C/min, which is nowhere near the limit. So this rules out the obvious failure modes like a method producing a fake spike, but passing this check doesn't mean the reconstruction is correct, just that it isn't physically impossible.

4.4 Engineering impact check (degree-hours)

The error metrics measure how far off a prediction is from the truth, but what actually matters for a real engineering decision is whether the number you feed into the next step of your workflow is right. For temperature that number is usually degree-hours which is the integrated area above a comfort threshold, which is what gets fed into chiller sizing, comfort compliance checks, and energy audits.

I used the trapezoidal rule cumulative_trapezoid from Week 6 to integrate the temperature excess above 20 °C over a 24h gap region. This is where the methods actually diverge the most, which is where it gets interesting. The true degree-hours over that region were 48.8 °C·h. The three interpolation methods all came out at 4.4 °C·h which is an underestimate of 91%, basically because they draw a near-flat line across the gap and completely miss the daytime peak. SVR landed at 49.5 °C·h, which is within 1.5% of the truth, and MLP overshot at 64.9 °C·h (33% too high).

This is a much stronger signal than MAE alone. A method with “reasonable” MAE can still get the integrated quantity catastrophically wrong, which means picking the wrong method could flip the sign of an engineering decision about whether a room is in compliance or not.

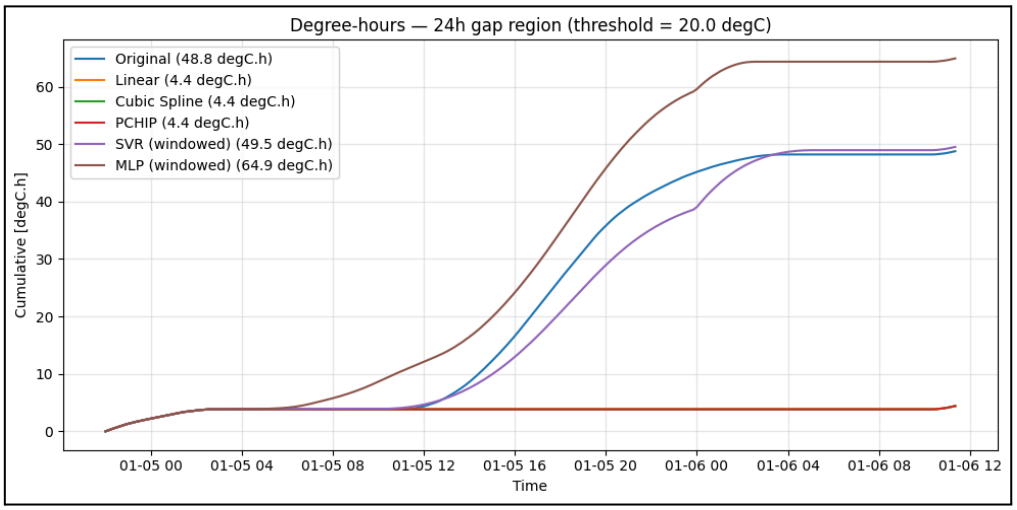


Fig 3: 24hr Gap Degree hour comparison for all 5 methods

4.5 Transfer validation on pressure

To check that the methodology works for more than just temperature I re-ran the full validation on the pressure channel from the same sensor. The ranking changed: PCHIP won with MAE 0.524 mb, Linear came second at 0.564 mb, and SVR dropped to third at 1.374 mb with a negative R2. The explanation is in the signal itself. Pressure is weather-driven and smooth with no diurnal cycle, so SVR has nothing periodic to learn from the 48h window and just adds noise. The lesson is that method choice depends on signal character, not variable name.

4.6 Results summary

Table 1: Best Method for 1hr, 6hr and 24hr (Based on Lowest MAE)

Gap size	Best method	MAE (degC)
1h	Linear Interpolation	0.146
6h	PCHIP	0.250
24h	SVR (RBF, windowed)	0.934

The overall ranking across all gap sizes, lowest MAE first, was: SVR 0.645, PCHIP 0.692, Linear 0.806, MLP 0.847, Cubic Spline 1.599, Mean Baseline 2.381. SVR beats the baseline by 72.9%, which is a large enough margin to say the ML windowing earns its added complexity.

The residual plots explain this ranking. For the 24h gaps, all three interpolation methods leave a smooth 5 to 10 degC hump across the gap region, and that hump is the diurnal cycle itself, the shape they cannot invent from the two endpoints they get to see. SVR’s residuals on the same gaps stay bounded within about 1.5 degC because the 48h training window gives it the daily cycle to learn from.

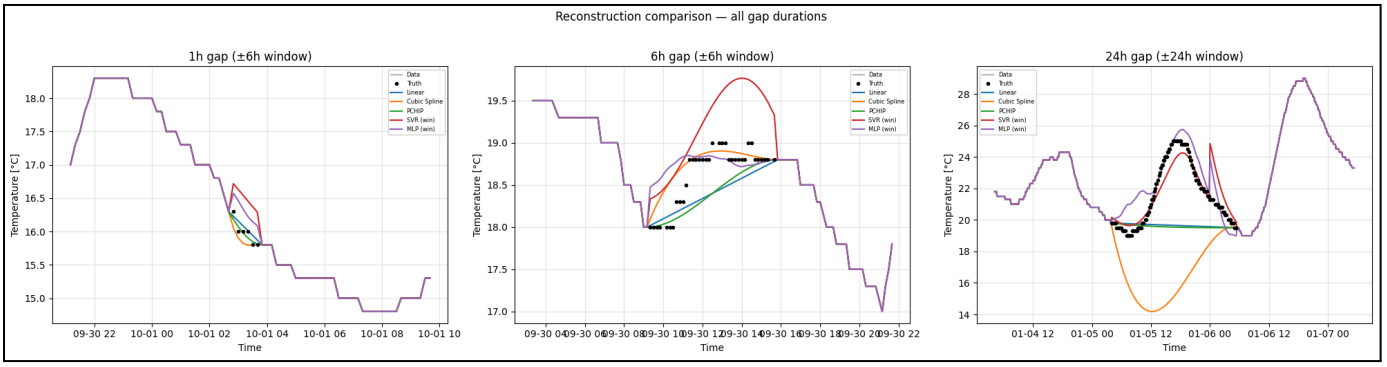


Fig 4: 1hr, 6hr and 24hr Reconstruction comparison for all 5 methods

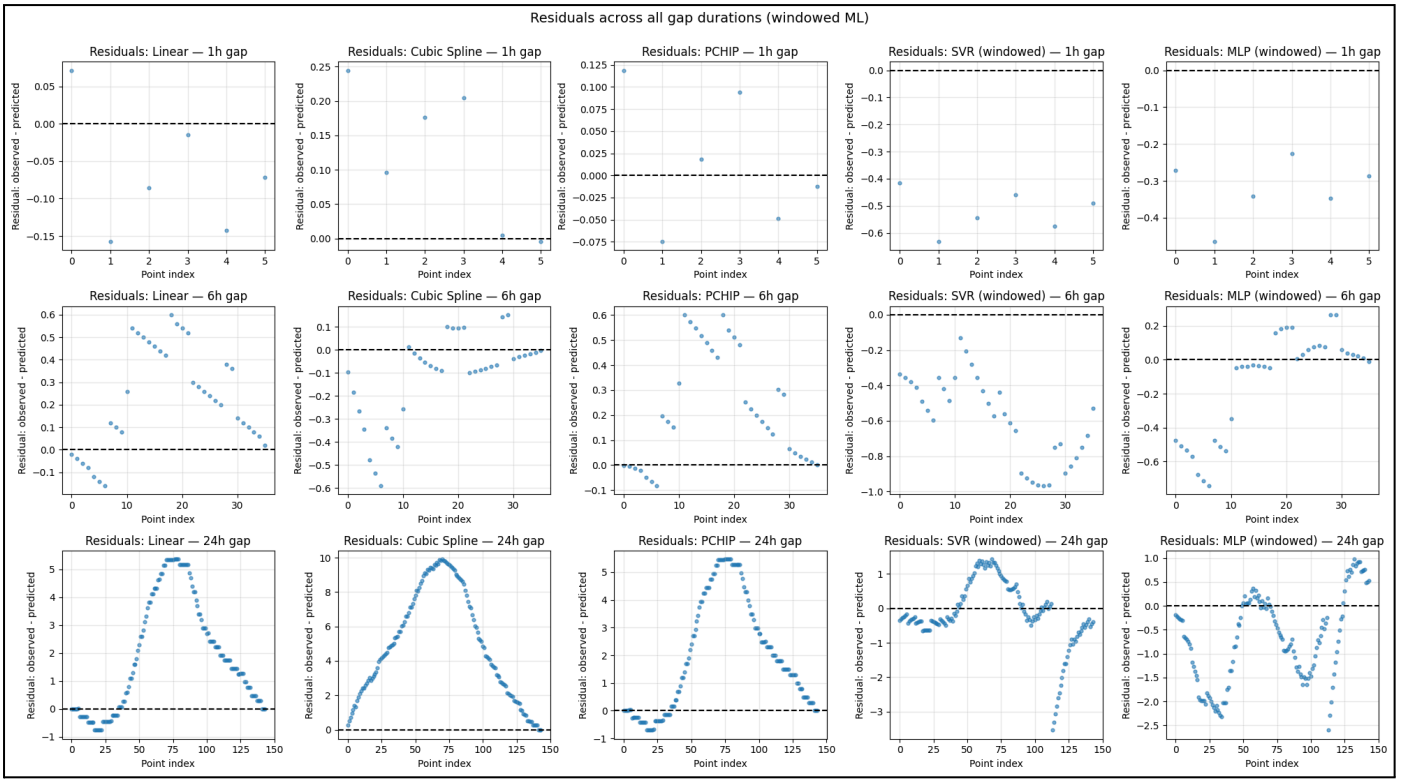


Fig 5: 1hr, 6hr and 24hr Residual Comparison for all 5 methods

4.7 What the validation cannot establish

The validation shows that for this one sensor in this one season, with gaps up to 24 hours, the method ranking I report is reliable. It does not establish how the methods behave on winter data with smaller diurnal amplitude, on gaps longer than a day, on sensors with different sampling intervals, or when a gap coincides with an unusual event such as a heatwave or a maintenance shutdown. The transfer test on pressure gives some reassurance that the methodology is not temperature-specific, but a single additional channel is not a strong basis for broader claims.

4.8 Did validation influence model selection

Yes, in two ways. The crossover between PCHIP and SVR across gap lengths pushed me away from recommending a single “best” method and toward a gap-length-dependent recommendation. The degree-hours result convinced me to add the integration check to the report, which was not in my original plan. **If I had only looked at MAE, I would have recommended SVR overall and missed the fact that PCHIP is both faster (0.003 s vs 0.012 s) and more accurate below 6 hours.**

5. Comparison and Optimisation Work

5.1 Multi-metric comparison

Table 2: Comparison across different metrics

Axis	Winner	Loser
Accuracy (MAE, 24h gap)	SVR (0.934)	Cubic Spline (4.09)
Runtime (24h gap)	Linear (0.002s)	MLP (0.960s)
Memory	All low (few MB)	None stand out
Interpretability	Linear, PCHIP	SVR, MLP
Implementation complexity	Linear (1 line)	MLP (hyperparameters, convergence)

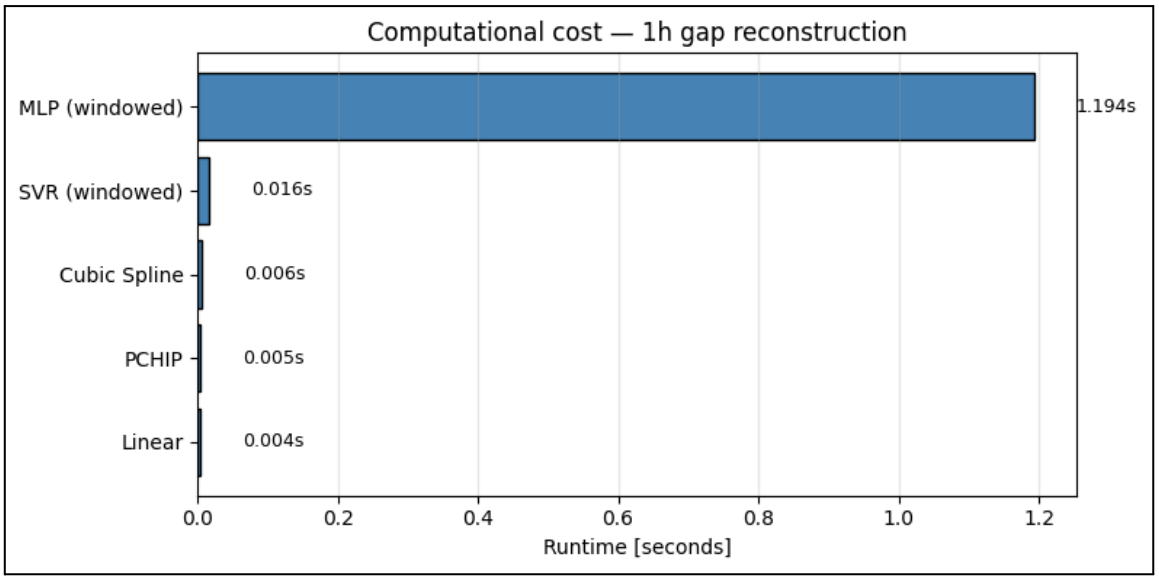


Fig 6: 1hr Gap Reconstruction Computational Cost comparison for all 5 methods

5.2 FLOPS estimates

Using the arithmetic-intensity formulae from Week 2, I estimated the floating-point operations for the three interpolation methods on the 24h gap (n_known = 28,894, n_gaps = 144). Linear needs 288 FLOPs (2 per gap point, no construction). PCHIP and Cubic Spline both need 232,592 FLOPs, dominated by construction (8 × n_known for the tridiagonal solve) with evaluation only 1,440 FLOPs. The ML methods are not amenable to a clean FLOPS estimate because the SVR kernel evaluations and the MLP forward/backward passes depend on internal state, so I report wall-clock runtime for those instead.

5.3 Optimisation: windowed ML

The original SVR implementation trained on the full 28,894 known points, took about 50 seconds per gap, and produced marginally worse accuracy than PCHIP on 24h gaps. After adding the windowing wrapper to restrict training to 48 hours around each gap (about 576 points), runtime dropped to about 12–15 ms per gap, roughly a 4,000-fold speedup, and accuracy on 24h gaps improved because the model stopped trying to fit irrelevant long-term

seasonal variation. This is the single most important optimisation in the project and it was motivated directly by the profiling runs.

5.4 Trade-offs and justification

For deployment I would recommend a hybrid: use Linear or PCHIP for gaps up to 6 hours (fast, interpretable, accurate), and windowed SVR for longer gaps where capturing the diurnal cycle matters. The MLP never wins on anything in this project (slower than SVR by 100x, no better accuracy) so I would not deploy it. Cubic Spline is unsafe on long gaps because of overshoot and should be dropped from consideration. The mean baseline is only there to provide the floor every real method must beat; it has no deployment role.

6. Limitations and Lessons Learned

Limitations

- **Single sensor used:** Three of the five sensors had firmware timestamp corruption, so only sensor 6012002000326 was usable. Results generalise to this one building zone, not to the full installation.
- **Winter data not covered:** The dense usable period runs September 2025 to April 2026, which includes spring, summer, and autumn in Melbourne but misses June through August. Cold-season behaviour with weaker diurnal amplitude is unexplored.
- **SVR hyperparameters not tuned:** C=10 and epsilon=0.5 are defaults carried straight from Week 5 and were not cross-validated. A proper grid search would likely improve 24h accuracy further.
- **Features are purely temporal:** The ML models only see hour of day and day of week. Adding neighbouring-sensor readings or external weather would be an obvious next step, but it starts to overlap with multi-sensor spatial modelling, which is better done as a separate project.
- **Degree-hours checked for one case only:** The engineering-impact result was computed for a single 24h gap at a 20 degC threshold. Sweeping other thresholds or applying it to the 1h and 6h gaps was not done, so this finding is strong for one case but not generalised.

Lessons learned

The biggest lesson was that error metrics and engineering impact are not the same thing. PCHIP looks fine on MAE and great on runtime, but when the integrated degree-hours matter it is 91% off. This changed how I would present reconstruction results in future work; any method ranking should be reported alongside the downstream engineering quantity it actually affects, not just the raw error.

The second lesson was that local information often beats more data. The windowed SVR trained on 50 times less data than the global version and still did better, because the irrelevant long-term variation was actively pulling the model away from the gap it was trying to fill. More data is not automatically better.

The third lesson was the value of auditing before modelling. If I had jumped into reconstruction with the raw dataset, I would have wasted time on three sensors that return the same timestamp for every reading. The 10 minutes I spent writing audit_data saved hours later.

7. AI-Use Reflection

I used Claude (Anthropic, Opus 4.7) and Google Colab's AI features throughout this project. All AI output was reviewed, tested, and verified by me before being kept, but the AI was involved in most parts of the work and I want to be clear about where.

What AI was used for. Scaffolding the module structure in src/, writing most of the first-draft code for each module, drafting docstrings in a consistent style, debugging Colab path and import errors, drafting the first version of the report sections, drafting the README, and helping me interpret the residual plots. The honest contribution level is high. I would say AI wrote the first draft of most text and code, and I rewrote, tested, verified, and in several cases restructured what it produced.

What stayed mine. The project idea and framing (gap reconstruction as an interpolation-vs-ML comparison), the choice of sensor (6012002000326, picked after auditing all five), the decision to pivot from a five-sensor study to one when the audit showed firmware corruption, the 48h window size for ML training, the 20 degC threshold for degree-hours, the decision to add the engineering-impact check to the report, the decision to keep the pressure transfer analysis after cutting PM2.5, and the overall structure of the validation. These are the choices that make this project rather than a generic reconstruction study, and they are mine.

Why I used AI. Honestly, it was faster. Writing boilerplate from scratch takes hours and I had other units running at the same time. Using AI for scaffolding freed me to spend more time on the parts that needed real thought: understanding the methods, interpreting the results, and deciding what mattered. I also used AI as a sanity check against the Week 1 to 7 course material, which was useful because it kept me going in the similar direction to what I was taught.

How I verified AI output. Every function was run end-to-end on real data before I kept it. Every number in the report (MAE, R2, degree-hours, runtime) was pulled directly from a notebook cell I ran myself, not reused from an AI claim. For the residual plot interpretation I zoomed in on the plots and confirmed the “diurnal hump” claim matched what I was seeing before writing it down.

Errors I made that I caught. The initial create_validation_gaps function used the first-two-point sampling interval to convert hours to samples, which gave all-NaN results on unevenly spaced data. I switched it to the median interval. The first SVR implementation trained on the full 29,000-point series at 50 seconds per gap, which was unusable at scale, and I added the windowing wrapper that cut it to about 15 ms.

Impact on my understanding. AI meant I spent less time on the mechanics of writing code and more time thinking about the methods and what the results meant. I understand the content well enough to defend every function and every number in the Q&A, which is the test that matters because AI is not available in the interview. If I were doing this project without AI I would have written less code and probably a shorter report, but the core choices (sensor, window size, threshold, scope) would have been the same because those came from the data, not from a tool.

8. References

1. American Society of Heating, Refrigerating and Air-Conditioning Engineers. (2023). Thermal environmental conditions for human occupancy (ASHRAE Standard 55-2023).
2. Fritsch, F. N., & Carlson, R. E. (1980). Monotone piecewise cubic interpolation. SIAM Journal on Numerical Analysis, 17(2), 238–246. doi.org
3. Monash University. (2026). MMA3001 Numerical methods and machine learning: Week 1 to 7 course notebooks
4. Monash University. (2026). Monash Smart Infrastructure Environmental Data
5. Scikit-learn developers. (n.d.). sklearn.svm.SVR, sklearn.neural_network.MLPRegressor, sklearn.pipeline.Pipeline, sklearn.preprocessing.StandardScaler. Scikit-learn documentation. scikit-learn.org
6. SciPy developers. (n.d.). scipy.interpolate.PchipInterpolator, scipy.interpolate.CubicSpline, scipy.integrate.cumulative_trapezoid. SciPy documentation. scipy.org